

TW Falcon: Triple-Word Arithmetic for Falcon

Giving Falcon the Precision to Fly Securely

Stef Halmans¹, Christine van Vredendaal², Tobias Schneider², Frank Custers² and Tim Güneysu¹

¹ Ruhr University Bochum, Bochum, Germany, stef.halmans@rub.de, tim.gueneysu@rub.de

² NXP Semiconductors, Eindhoven, Netherlands, christine.cloostermans@nxp.com,
tobias.schneider@nxp.com, frank.custers_1@nxp.com

Abstract. The post-quantum signature scheme Falcon is an attractive scheme for constrained devices due to its compactness and verification performance. However, it relies on floating-point arithmetic for signature generation, which - alongside physical security concerns - introduces two additional drawbacks:

Firstly, if implemented using the standard double-precision format, Falcon does not satisfy the formally proven error bounds required for a secure Gaussian sampler implementation. Although no practical attacks exploiting this limitation are currently known, it does give future attack concerns. Secondly, when looking at constrained devices, 32-bit constrained devices can lack hardware support for high-precision floating-point arithmetic and its use introduces significant performance overhead, as it must be emulated using integers.

In this work we present a novel method to address these limitations: We show that Falcon can be implemented using *single-precision* floating-point numbers. Our proposed method uses Triple-Word Floating-Point (TW) arithmetic and achieves a precision of at least 72 bits, compared to the 53 bits of double-precision floating-point arithmetic. We show our implementation achieves error bounds that meet the formal security requirements for a secure Gaussian sampler implementation, while maintaining other security guarantees. This way, Falcon can run on constrained devices equipped only with a single-precision Floating-Point Unit (FPU) without the need for integer emulation.

We demonstrate the feasibility of our approach on the Nucleo-L4R5ZI board, which features a Cortex-M4F processor enabled with a single-precision FPU. More precisely, we show the cost of increasing the precision of Falcon in this way only increases the computational effort by a factor of approximately 1.84 compared to the CPU cycles required for an implementation using emulated double-precision arithmetic via integers.

Keywords: Falcon · FN-DSA · triple-word floating-point arithmetic · constrained devices · post-quantum cryptography

1 Introduction

Steady progress in quantum computing research poses a significant threat to classical public-key cryptography, which are based on integer factorization and discrete logarithms. In response to this threat, the National Institute of Standards and Technology initiated the Post-Quantum Cryptography (PQC) Standardization Project to identify and standardize quantum-resistant algorithms [NIS]. As of today, NIST has identified four algorithms for standardization. The first three, key encapsulation mechanism ML-KEM [SAB⁺22], and digital signature schemes ML-DSA [LDK⁺22] and SLH-DSA [HBD⁺22] have been standardized as FIPS-203 [NIS23b], FIPS-204 [NIS23a] and FIPS-205 [NIS23c] respectively.

The fourth digital signature scheme Falcon [PFH⁺22] is currently being drafted as a standard under its new name FN-DSA and target of this paper. Falcon was selected by NIST for its favorable balance of performance, security, and implementation compactness, making it seemingly a promising candidate for deployment in constrained environments such as embedded systems and IoT devices.

Despite its benefits for embedded systems, Falcon also poses considerable drawbacks for these platforms. Most prominently, Falcon relies on double-precision floating-point arithmetic for its key generation and signature operations. Firstly, not all embedded systems are able to perform double-precision floating-point arithmetic. Larger constrained devices often feature an Floating-Point Unit (FPU) for these operations, but highly constrained ones typically do not. Devices based on a Cortex-M4 or Cortex-M33 have an optional single-precision FPU, while for higher-end processors like the Cortex-M7 a single-precision FPU is always available, but the double-precision FPU is optional [Yiu16, Arm25]. Vendors can therefore choose to design in Cortex-M7, but still disable the use of double-precision for code-size, performance, or power consumption reasons.

Secondly, even the double-precision FP arithmetic is not sufficient for the security proof of the Falcon digital signature scheme. Recall that in [PFH⁺22] the security proof only holds, i.e. resisting attacks of 2^{64} signature queries, when the double-precision errors are bound by 2^{-46} . [PFH⁺22] shows the values can in fact go up to 2^{-40} and thereby only ensuring protection against 2^{53} queries. In this work we will show that in practice the error values even go up to 2^{-37} , which gives concern for attack avenues escaping the proof guarantees.

One direction is to switch out the double-precision operations: [PFH⁺22] suggests to emulate all floating-point operations (such as FFTs and Gaussian sampling) using integer arithmetic implementing the IEEE 754 binary64 operations [IEE19]. This conversion significantly slows down performance [Por19, Por25a], but can then be run on devices without a double-precision FPU. This, however, does not impact security. We will compare the performance of our implementation against [Por25a].

Triple-Word Floating-Point (TW) arithmetic [FMP19] are algorithms designed to extend the precision of numerical computations by representing a high-precision value as the sum of three lower-precision floating-point numbers. By decomposing a value into three components (i.e., the high, middle, and low words) triple-word arithmetic enables more precision and therefore more accurate arithmetic. Consequently, this approach yields reduced floating-point errors, leading to a Gaussian sampler implementation that abides by the proven error bounds.

Contributions. In this paper we observe that for embedded devices that have a *single-precision* FPU available, Falcon can be implemented securely with a performance penalty of 85% compared to integer-emulated Falcon. By representing higher precision floating-point values in three single-precision floating-point values, Falcon signature generation can be performed at a lower error rate. To the best of our knowledge, we are the first to apply triple-word arithmetic to a cryptographic scheme.

Because our approach only requires a single-precision FPU compared to the double-precision FPU for the original implementation, it can be enabled on a larger range of embedded devices. It offers a practical balance between computational efficiency, numerical accuracy, and the security of Falcon, making it a compelling choice for high-assurance software and hardware implementations of Falcon.

More concretely, in this work:

- We are the first to apply and analyze the application of triple-word arithmetic to a cryptographic scheme, using three single-precision floating-point values for combined precision. To make it fully applicable to the Falcon digital signature

scheme, we develop multiple new triple-word arithmetic operations that were not covered in [FMP19].

- We show that the security proof gap pointed out in [PFH⁺22] is in fact larger than implied. The security proof bounds the floating-point error of the Gaussian sampler at 2^{-46} . Based on experimental data, it was claimed that the double-precision standard achieves a floating-point error bound for the Gaussian sampler of 2^{-40} , i.e., 6 bits less than required for the security proof. We show that this error bound is actually 2^{-37} , i.e., 9 bits less than required by the proof. Although this has not been utilized in a practical attack, this deviation from a Gaussian distribution gives concern for future exploitation, as in theory, with the current security proof, fewer than 2^{64} signature queries are allowed, contrary to the NIST PQC requirements.
- We offer a *constant-time* implementation of Falcon using triple-word arithmetic. This implementation contains both security levels and is created for the Cortex-M4.
- We analyze the practical rounding errors leading to the security proof gap in current implementations of Falcon in comparison to our new implementation. We show that ours gives sufficient security guarantees, while the existing does not.
- We show that the performance impact of our implementation compared to the heavily optimized integer-emulated implementation by [Por25a] is on average less than 85% for both Falcon parameter sets, while having error values that are much better than required by the proof. The integer-emulated FP arithmetic uses a 64-bit integer to emulate the 64-bit floating-point standard (double-precision).

Outline. The remainder of this paper is structured as follows. Section 2 gives an overview of the Falcon scheme, floating-point arithmetic and the core algorithms of triple-word arithmetic. In Section 3 the security proof for the Gaussian sampler in Falcon is analyzed, precision tests are recreated, and its resulting implications are discussed. Section 4 presents how to use triple-word arithmetic in Falcon. We show how the core algorithms can be applied, introduce new algorithms, and adapt all TW algorithms to be constant-time for additional security. In Section 5 we analyze the implementation. We show its improved error values compared to double-precision and integer emulated Falcon, as well as evaluate the performance impact of using TW arithmetic in Falcon. Finally, Section 6 concludes the paper. We summarize our results and discuss its security implications, as well as theorize how integer emulation could be adapted to achieve higher precision.

2 Background

In this section, we first introduce the Falcon digital signature scheme. Then we discuss the necessary theory on floating-point numbers and Triple-Word Floating-Point (TW) arithmetic.

2.1 Falcon

The signature scheme Falcon [PFH⁺22] (to be standardized as FN-DSA) is a lattice-based hash-and-sign scheme, based on the GPV framework which is proven secure under the quantum oracle model [GPV08, BDF⁺11].

In short, the GPV framework uses a public matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ defining a q -ary lattice Λ_q , and a secret trapdoor basis $\mathbf{B} \in \mathbb{Z}_q^{m \times m}$ for the orthogonal lattice $\Lambda_q^\perp$ ($\perp$ meaning orthogonal), with $\mathbf{B} \times \mathbf{A}^T \equiv 0 \pmod{q}$.

Table 1: Average number of invocations of floating-point operations in one Falcon-512 signature generation [Por25a].

Operation	fpr_of	add	add_sub	mul	div	sqrt
Invocations	11,992	62,680	32,768	106,685	2,304	512

To sign a message x , one first computes a preimage $\mathbf{c}_0 \in \mathbb{Z}_q^m$ such that $\mathbf{c}_0 \mathbf{A}^T = H(x)$ for some hash function $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q^n$. Using $\mathbf{B}$, a nearby lattice vector $\mathbf{v} \in \Lambda_q^\perp$ is sampled from a discrete Gaussian over $\Lambda_q^\perp$, and the signature is $\mathbf{s} = \mathbf{c}_0 - \mathbf{v}$. Verification checks that $\mathbf{s} \mathbf{A}^T = H(x)$ and that $\mathbf{s}$ is “short”. The security relies on the hardness of finding such short solutions without the trapdoor basis. The GPV framework suggests using Klein’s sampler to sample from a discrete Gaussian. This is a randomized version of Babai’s nearest plane algorithm [Kle00, Bab86].

Falcon is based on the NTRU lattice structure and the signature generation mostly operates in the Fast Fourier Transform (FFT) domain to accelerate polynomial arithmetic. Discrete Gaussian sampling is performed using the `ffSampling` algorithm based on [DP16]. This algorithm constructs a tree structure by recursively splitting polynomials into smaller subgroups, similar to how the FFT decomposes polynomials for multiplication. At each node of the tree, an $\mathbf{LDL}^*$ decomposition ([GVL13, Chapter 4.1.2]) is applied, which is later used to combine the results from the child nodes into the final sampled lattice point. This process produces a Falcon tree, where the polynomials at the leaves are reduced to constants. At the leaves, these constants serve as the mean and standard deviation for sampling values from a discrete Gaussian distribution. The sampled values from the leaves are then propagated up the tree by merging them, together with the $\mathbf{LDL}^*$ decomposition results, to reconstruct the lattice point in the original, higher-dimensional space. Essentially, this algorithm is a fast Fourier variant of Babai’s nearest plane algorithm.

Arithmetic with real numbers is required throughout this process. Splitting and merging in the Falcon tree requires floating-point multiplication and addition operations. The $\mathbf{LDL}^*$ decomposition and Gaussian sampling require division of floating-point numbers. Additionally, each leaf node is normalized, which involves both a square root and a division operation. Of these, the square root is used least frequently (once per leaf), division is used once per node and twice per leaf, while addition and multiplication are used extensively at every node and leaf, and thus dominate the computational workload. In Table 1, the average number of operation invocations is shown for Falcon-512, showing that addition and multiplication are used a lot more than division and square root. The operation `fpr_of` converts an integer to a floating-point value, and the operation `add_sub` computes an addition and subtraction of the same inputs, an operation frequently used in the FFT.

The floating-point operations can be emulated with integer representations [Por19, Por25a], using a 64-bit integer that emulates a 64-bit floating-point value, but this slows down the performance significantly. As this is not the focus of this paper, we will omit those technical details here.

2.2 Floating-Point Numbers

Falcon relies on real numbers in its signature generation algorithm. The dominant format used to represent real numbers in computing systems today is the IEEE 754 Floating-point standard [IEE19]. Herein numbers are represented as:

$$\pm d_0.d_1d_2\dots d_{p-1} \times \beta^e,$$

where β is the base, d_i is a digit in base β , p is the precision, and the exponent e is within the range $[e_{\min}, e_{\max}]$. In this paper we will assume the base $\beta = 2$. The binary layout of a floating-point numbers follows a structured format, as shown in Figure 1. The first bit

Table 2: IEEE 754 precision and exponent properties.

Format	Precision Bits	Exponent Bits	Exponent Min	Exponent Max
binary32	24	8	-126	+127
binary64	53	11	-1022	+1023

represents the sign of the value. Then, bits e_i represent the exponent value, computed as $e = |e_0 \dots e_{n-1}|_2 - b$ where b is a bias making sure both positive and negative exponents can be represented. The last bits d_i represent the mantissa, storing the significant digits of the number.



Figure 1: General binary floating-point representation, where every block is a bit and the lowest bit is on the right.

In Table 2 the details about the single-precision floating-point standard, binary32, and the double-precision floating-point standard, binary64, are shown. The IEEE 754 standard defines several special values that handle edge cases in floating-point arithmetic, but for Falcon only the case of zero is important. The ± 0 is represented by setting all exponent and fraction bits to zero. The sign bit distinguishes between $+0$ and -0 .

Due to the finite precision, errors can appear in floating-point arithmetic. Rounding errors occur when numbers that cannot be exactly represented are rounded to the nearest IEEE 754-representable value, introducing small discrepancies. Cancellation errors happen when subtracting two nearly equal numbers, causing a loss of significant digits and reducing precision. Small floating-point errors can compound over multiple operations, ultimately leading to noticeable inaccuracies.

The relative error measures how much a floating-point approximation deviates from the true value:

$$\text{Relative Error} = \frac{|x - x_f|}{|x|},$$

where x is the true value and x_f is its floating-point representation. Machine epsilon (ϵ) defines the upper bound on the relative error when rounding a real number to the closest floating-point representation.

The Unit in the Last Place (ULP) is defined as the gap between two consecutive floating-point numbers. The relative error of a representation is therefore bounded by half the ULP.

Additional accuracy for floating-point representation can be gathered by adding extra bits of precision. For single-precision (binary32 in IEEE 754) floating-point numbers, the machine epsilon is $\epsilon_{\text{single}} = 2^{-23} \approx 1.19 \times 10^{-7}$, which corresponds to roughly 7 decimal digits of accuracy. In double-precision (binary64 in IEEE 754), $\epsilon_{\text{double}} = 2^{-52} \approx 2.22 \times 10^{-16}$, providing about 16 decimal digits of accuracy.

2.3 Triple-Word Floating-Point Algorithms

Adding extra bits to increase precision is not always an option for constrained devices. As an alternative, multiple floating-point values can be combined to compute arithmetic with higher precision (see e.g. [Pri91]).

Algorithms optimized for the use of three floating-point values to gain higher precision are specified by [FMP19]. These algorithms work for any implementation of floating-point

numbers, such as the IEEE 754 binary32 format. For this work we focus on emulating the 53 bits of precision in the IEEE 754 binary64 format using three values in the binary32 format, giving a combined precision of 72 bits. In [FMP19], algorithms for addition, multiplication, reciprocal, division, and square root are specified.

A Triple-Word Floating-Point (TW) value is defined as a triplet (x_0, x_1, x_2) of floating-point numbers that is *P-nonoverlapping*. The sequence (x_i) is P-nonoverlapping when $\forall i, |x_{i+1}| < \text{ulp}(x_i)$, where $\text{ulp}(x_i)$ is the unit in the last place defined as $\text{ulp}(x) = |\text{next floating-point number} - x| = 2^{\lfloor \log_2 |x| \rfloor} \cdot 2^{-p+1}$ for precision p . The sequence (x_i) is additionally F-nonoverlapping when $\forall i, |x_{i+1}| < \frac{1}{2} \text{uls}(x_i)$, where uls is the unit in the last significant place, defined as the largest power of 2 that divides x_i .

As a basis for these full TW arithmetic algorithms, two main building blocks are used to compute the sum and product of single floating-point numbers (2Sum, 2Prod). These take two single floating-point numbers as input and return two floating-point numbers, one representing the result of the computation (sum resp. product) and the other the error of the computation. Additionally, a Fused Multiply-Add (FMA) operation should be available, that multiplies two floats, and adds the result to another float, without intermediate rounding.

To make sure the result of a computation is a TW value, the algorithms VecSum and VSEB are used. VecSum turns a sequence of “slightly” nonoverlapping floating-point numbers into a “more” nonoverlapping sequence. It does this according to the diagram shown in Figure 2a and ensures that the sum of the output vector is the same as the sum of the input vector. The output vector of VecSum is F-nonoverlapping if and only if the input vector is sorted from biggest to smallest. Then, the algorithm VSEB puts an F-nonoverlapping input vector into a P-nonoverlapping vector. An illustration of this algorithm is visible in Figure 2b.

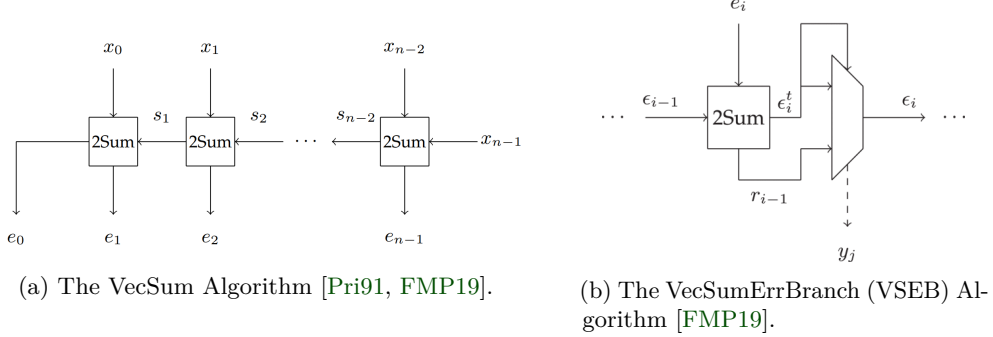


Figure 2: TW base algorithms.

We will not repeat all algorithms from [FMP19] here, but instead highlight the most important ones: the sum (Algorithm 1) and product (Algorithm 2 (3Prod)) of two TW values. In Algorithm 1, the values of two TW numbers are combined and sorted from biggest to smallest (Merge, absolute values are compared). VecSum and VSEB are then used to add and convert this sorted input into a sequence of 6 P-nonoverlapping floating-point numbers, of which the first three represent the TW. This essentially computes the sum of the two TW inputs ensuring a proven computation error bound. The number $u = \frac{1}{2} \text{ulp}(1) = 2^{-p}$ denotes the roundoff error unit of floating-point operations, where p is the amount of precision bits.

We will illustrate the addition of two TW numbers, $\hat{x} = [1.12, 1.23e^{-3}, 5.46e^{-6}]$ and $\hat{y} = [2.12, 3.11e^{-4}, 7.33e^{-6}]$, in a toy floating-point system with three decimal digits of precision. The first step is to merge them together, by sorting the absolute values from biggest to smallest. This results in the list $[2.12, 1.12, 1.23e^{-3}, 3.11e^{-4}, 7.33e^{-6}, 5.46e^{-6}]$.

Table 3: Example of VecSum computation, each row is a 2Sum call, computing $x + y = s + e$, where s is then used in the next row to add together with the next input.

x	y	s	e
$7.33e^{-6}$	$5.46e^{-6}$	$1.28e^{-5}$	$-1e^{-8}$
$3.11e^{-4}$	$1.28e^{-5}$	$3.24e^{-4}$	$-2e^{-7}$
$1.23e^{-3}$	$3.24e^{-4}$	$1.55e^{-3}$	$4e^{-6}$
1.12	$1.55e^{-3}$	1.12	$1.55e^{-3}$
2.12	1.12	3.24	0

Next, we apply the VecSum algorithm to this list. It starts from right to left, repeatedly calling 2Sum to produce a rounded sum s and an error e with $x + y = s + e$, carrying s forward and storing e , as illustrated in Figure 2a. This computation for our example can be found in Table 3, resulting in the list $[3.24, 0, 1.55e^{-3}, 4e^{-6}, -2e^{-7}, -1e^{-8}]$, which satisfies the F-nonoverlapping definition.

We then apply VSEB to renormalize this expansion into a shorter, well-formed one. VSEB maintains a small accumulator ϵ and scans the list, repeatedly computing $r_i, \epsilon_{i+1} = 2Sum(\epsilon_i, e_{i+1})$. If $\epsilon_{i+1} \neq 0$, it emits r_i as the next output component and carries ϵ_{i+1} forward, else it carries r_i forward without emitting an output. This follows the diagram in Figure 2b. There is a final 2Sum call, which splits the remaining tail into the last two outputs. This preserves exactness, i.e., the sum of outputs equals the original total, while producing a compact, P-nonoverlapping expansion. The first three components represent the TW sum. Any additional non-zero components capture precision beyond the TW format and will be discarded. The expansion created in VSEB would be $[3.24, 1.55e^{-3}, 3.79e^{-6}, 0, 0, 0]$, resulting in the final result $\hat{z} = \hat{x} + \hat{y} = [3.24, 1.55e^{-3}, 3.79e^{-6}]$.

Algorithm 1 TWSum($x_0, x_1, x_2, y_0, y_1, y_2$).

Require: $\bar{x}, \bar{y} \in \text{TW}$

Ensure: $\bar{r} \in \text{TW}$ and $\left| \frac{\bar{r} - (\bar{x} + \bar{y})}{\bar{x} + \bar{y}} \right| \leq 2u^3 + 4.2u^4$

- 1: $z_0, \dots, z_5 \leftarrow \text{Merge}((x_0, x_1, x_2), (y_0, y_1, y_2))$
 - 2: $e_0, \dots, e_5 \leftarrow \text{VecSum}(z_0, \dots, z_5)$
 - 3: $r_0, r_1, r_2 \leftarrow \text{VSEB}(3)(e_0, \dots, e_5)$
 - 4: **return** (r_0, r_1, r_2)
-

Algorithm 2 (3Prod) computes the product between two TW numbers $\hat{x} = (x_0 + x_1 + x_2)$ and $\hat{y} = (y_0 + y_1 + y_2)$ by expanding the product and then carefully organizing the dominant terms so that the final result fits back into a well-formed three-component expansion. In full, $(x_0 + x_1 + x_2) \cdot (y_0 + y_1 + y_2)$ contains 9 cross terms. The smallest three terms, x_1y_2 , x_2y_1 , x_2y_2 , are below the precision we intend to keep. Consequently, we only compute the six more significant terms:

$$x_0y_0, x_1y_0, x_0y_1, x_1y_1, x_2y_0, x_0y_2.$$

First, the three largest cross terms are computed with 2Prod, which for each pair returns a rounded product z_{ij}^+ and an error z_{ij}^- such that $x_iy_j = z_{ij}^+ + z_{ij}^-$. The three pieces that may overlap in magnitude, namely z_{00}^- , z_{01}^+ and z_{10}^+ , are then collected by a single VecSum call to produce (b_0, b_1, b_2) . At this point we still need to incorporate x_1y_1 , x_0y_2 , and x_2y_0 . The product x_1y_1 can overlap with b_2 , and x_0y_2 and x_2y_0 can overlap with z_{10}^- and z_{01}^- , respectively. However, for these three products we do not need to retain their error results to reach TW precision. We therefore use Fused Multiply-Add (FMA) to combine each product with its accumulator in a single rounding (line 5, 6, 7). The last two are summed

once more (line 8). After these steps, we have five partially ordered pieces that together approximate the full product within the desired accuracy, z_{00}^+ , b_0 , b_1 , c , and z_3 . To form a correct TW result, they have to be put in a P-nonoverlapping sequence, similar to the sum algorithm. First VecSum is used, from which the first output is seen as r_0 , and the others are put through VSEB, giving r_1 and r_2 , resulting in the TW value $\hat{r} = \hat{x} \cdot \hat{y} = (r_0, r_1, r_2)$.

Algorithm 2 $3\text{Prod}_{3,3}^{acc}(x_0, x_1, x_2, y_0, y_1, y_2)$

Require: $\bar{x}$ and $\bar{y}$ TW; $p \geq 6$

Ensure: $\bar{r}$ TW and $\left| \frac{\bar{r} - \bar{x}\bar{y}}{\bar{x}\bar{y}} \right| \leq 28u^3 + 107u^4$

- 1: $z_{00}^+, z_{00}^- \leftarrow 2\text{Prod}(x_0, y_0)$
 - 2: $z_{01}^+, z_{01}^- \leftarrow 2\text{Prod}(x_0, y_1)$
 - 3: $z_{10}^+, z_{10}^- \leftarrow 2\text{Prod}(x_1, y_0)$
 - 4: $b_0, b_1, b_2 \leftarrow \text{VecSum}(z_{00}^-, z_{01}^+, z_{10}^+)$
 - 5: $c \leftarrow \text{RN}(b_2 + x_1 y_1)$ (FMA)
 - 6: $z_{3,1} \leftarrow \text{RN}(z_{10}^- + x_0 y_2)$ (FMA)
 - 7: $z_{3,2} \leftarrow \text{RN}(z_{01}^- + x_2 y_0)$ (FMA)
 - 8: $z_3 \leftarrow \text{RN}(z_{3,1} + z_{3,2})$
 - 9: $e_0, e_1, e_2, e_3, e_4 \leftarrow \text{VecSum}(z_{00}^+, b_0, b_1, c, z_3)$
 - 10: $r_0 \leftarrow e_0$
 - 11: $r_1, r_2 \leftarrow \text{VSEB}(2)(e_1, e_2, e_3, e_4)$
 - 12: **return** (r_0, r_1, r_2)
-

Note that [FMP19] has a “fast” version of Algorithm 2, which computes intermediate values with less precision resulting in a lower proven error bound. Because of some pre-knowledge of input orders/values, the fast version of 2Sum can be used at some places in the TW algorithms. The details of this, the algorithms of other operations, as well as when the fast version of 2Sum can be used, can be found in [FMP19].

3 Falcon Precision Proof

Falcon relies on Gaussian sampling for its signature generation. In practice, sampling from an exact Gaussian distribution $\mathcal{D}$ is not feasible due to the reliance on floating-point arithmetic. As discussed in Section 2.2, floating-point numbers have limited precision, which introduces rounding errors. Consequently, the practical distribution $\mathcal{D}_\delta$ will be different from the exact distribution $\mathcal{D}$, where $\mathcal{D}_\delta$ is close to $\mathcal{D}$ parameterized by δ .

The extent of the difference depends on the precision of the floating-point arithmetic used. [Pre17] shows a method to use the Rényi divergence to get a tighter bound (compared to previous work) for the difference between the real and estimated distributions for Klein’s sampler. This results in an estimated precision of 61 bits for $\lambda = 256$. Falcon uses a fast Fourier variant of Klein’s sampler, thus they use a similar approach to the analysis of Klein’s sampler done in [Pre17]. In this section the idea behind the analysis in Falcon is explained. Then, a closer look at the precision analysis results is taken to figure out what the result of the precision experiments done in Falcon mean for security. These experiments are recreated on the latest Falcon reference code, and show a different result than stated in the Falcon specification [Por25a, PFH⁺22]. From these experiments, a conclusion is drawn for how many precision bits the Gaussian sampler is provably secure. Lastly, to get enough precision to abide by the proven upper bound, the use of TW is given as an alternative to the IEEE 754 double-precision floating-point values, which is investigated more in Section 4.

3.1 Formal Error Evaluation

The Falcon specification suggests using the double-precision floating-point standard from IEEE-754 [IEE19]. To show that this standard gives enough precision, they use a Rényi divergence-based analysis of Klein’s sampler. This analysis relies on Lemma 3 and Equation 4 from [Pre17, Section 3.2, 3.3], where from Lemma 3 we know that

$$\exists \delta > 0, \text{ s.t. } 1 - \delta \leq \frac{D_\delta}{D} \leq 1 + \delta.$$

Lemma 3 and Equation 4 can be combined to get the following bound on δ :

$$\delta^2 \leq \frac{1}{4\lambda q_s}. \quad (1)$$

This, for $q_s = 2^{64}$ and $\lambda = 256$, results in $\delta \leq 2^{-37}$ [Pre17].

In Falcon there are $2n$ discrete Gaussian samplers denoted by $\mathcal{D}_{\mathbb{Z}, c_j, \sigma_j}$. The floating-point values c_j (resp. σ_j) are known with an *absolute* error (resp. *relative* error) at most δ_c (resp. δ_σ). For any output of the Gaussian sampler used by Falcon with non-negligible probability, we have $\delta \leq 20n(\delta_c + \delta_\sigma)$ in the worst case [PFH⁺22]. This delta is from Lemma 3. Additionally, they improve this bound for the average case, by replacing the factor $2n$ with $\sqrt{2n}$, resulting in $10\sqrt{2n}(\delta_c + \delta_\sigma)$.

To derive the bound on the combined errors $(\delta_c + \delta_\sigma)$, we use the fact that when the Relative Error Lemma (Lemma 3) applies, for $\lambda = 256$ and $q_s = 2^{64}$, we have $\delta \leq 2^{-37}$. We can now compute the bound using the equation

$$10 \cdot \sqrt{2n} \cdot (\delta_c + \delta_\sigma) \leq 2^{-37},$$

resulting in $(\delta_c + \delta_\sigma) \leq 2^{-46}$.

The Falcon specification mentions that experimental results of the signature scheme, implemented using the double-precision standard, show they always have $(\delta_c + \delta_\sigma) \leq 2^{-40}$, which is worse than the required bound of $\leq 2^{-46}$. To understand the security implications of this difference, we go back to Equation 1, and we substitute the value 2^{-40} for $(\delta_c + \delta_\sigma)$. We then obtain the following inequality:

$$\begin{aligned} (10 \cdot \sqrt{2n}(\delta_c + \delta_\sigma))^2 &\leq \frac{1}{4\lambda q_s} \\ 2^{-61} &\leq \frac{1}{\lambda q_s}. \end{aligned}$$

If we want to keep security of the highest security level, we have to use $\lambda = 256$, which results in the following inequality:

$$2^{-53} \leq \frac{1}{q_s}.$$

From this, we can conclude that if $(\delta_c + \delta_\sigma) \leq 2^{-40}$ and $\lambda = 256$, the imperfect distribution based on the double-precision standard is only proven to be secure up to 2^{53} queries, while the NIST requirement is 2^{64} . This indicates that for the Rényi based security argument, the double-precision standard does not provide enough precision. Note, the Falcon specification only states that the values always stay below 2^{-40} , not if this is close to the average case or just a far outlier.

To better understand the actual distribution of error values, and to verify the claim that the combined error consistently stays below 2^{-40} , the precision test was recreated. This analysis will be presented in Section 3.2.

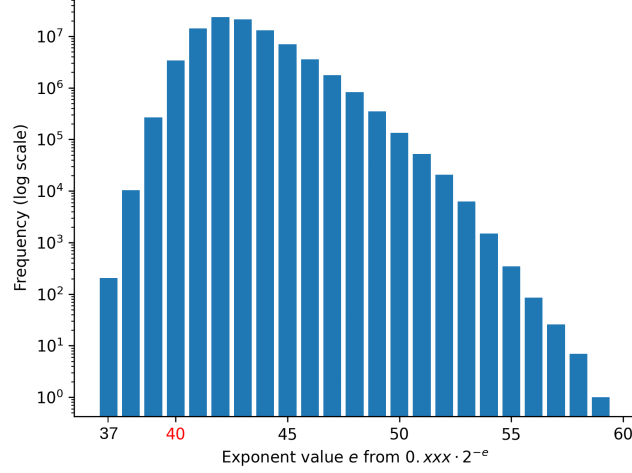


Figure 3: The frequency of each exponent of the combined error value $(\delta_c + \delta_\sigma)$ for Falcon-1024 from 100,000 signature generation runs.

3.2 Precision Testing in Practice

In the Falcon specification, it is stated that the test for error values was done by running the signature scheme twice: once using floating-point arithmetic with 53 bits of precision, and once with 200 bits of precision. The version with 200-bit precision is treated as having “infinite precision”, and serves as a reference to compute the error value of the 53-bit precision version.

To implement this in the latest Falcon code, [Por25b], the MPFR library is used [FHL⁺07]. This library can be used to perform floating-point arithmetic with arbitrary precision. All arithmetic performed in the Falcon code is modified to perform a high-precision and low-precision computation. Additionally, all constants in the code are recalculated to support computations with high-precision values. As explained in Section 3.1, the errors that are important are from the mean (μ) and the standard deviation (σ), which are the inputs to the Gaussian sampler. So, for these values the errors are computed by comparing the low-precision value (treated as the computed result) with the high-precision value (treated as the “real” value). Specifically, the absolute error is calculated for the parameter μ , and the relative error is calculated for σ , or more precisely, for σ^{-1} , since σ itself is never computed in the code for efficiency reasons. The value μ is derived from the splitting of the target polynomial t down the Falcon tree. Polynomial t is computed using the inverse of the private key and the hashed message. In contrast, σ corresponds to a leaf in the Falcon tree, and is obtained from the splitting of the private key. This error computation is done for integer emulation, double type, and 53-bit MPFR values to check if this would influence the values, which as expected, it did not.

Then, the combined error $(\delta_\mu + \delta_\sigma)$ is computed. Note, the value μ is denoted by the value c in the precision evaluation proof in Falcon, and this notation is kept in Section 3.1. Rather than storing the full floating-point values, only the exponent e is saved, where the value is represented in the form $0.x_1x_2\dots x_n \cdot 2^{-e}$, with each digit x_i satisfying $0 \leq x_i \leq 9$. To get enough data points, the signature generation part of Falcon-1024 was executed 100,000 times, each time using a different randomly generated messages and key. The results are presented in Figure 3, which shows a bar plot of the frequency distribution of the error exponents observed across the 100,000 runs.

The Falcon specification claims that from their tests, the combined error always satisfies $\leq 2^{-40}$, which is already worse than the proven $\leq 2^{-46}$, as discussed in Section 3.1. In Figure 3, this exponent value 40 is highlighted in red. The graph shows that this claim does not hold: despite logarithmic scale, a significant number of errors fall at lower exponent values, even as low as 37. From these tests, it can be concluded that for 100,000 Falcon-1024 runs, the combined error is always below 2^{-37} . This is 3 bits worse than the claimed bound, which was already 6 bits worse than the theoretical requirement resulting from the proof.

Empirical testing, even with 100,000 signatures, cannot prove that a bound will never be exceeded. These runs yield about 102.4 million leaf instances of μ and σ , from which the error values are computed. As the sample size grows, the chance of observing rarer extremes increases; for example, with 1 million signatures one could plausibly see an error exponent as low as -36 . This may also explain why earlier reports (e.g., [PFH⁺22]) appear more favorable: they likely used fewer trials and thus observed fewer extremes. That said, the empirical frequency of error exponents decays roughly exponentially, so very low exponents are highly unlikely, though not impossible, in practice.

As in Section 3.1, we can analyze the theoretic security implications of this result. When using Equation 1 with $(\delta_c + \delta_\sigma) = 2^{-37}$ and $\lambda = 256$, we obtain the following inequality:

$$2^{-47} \leq \frac{1}{q_s}.$$

This implies that, under a worst-case analysis using Rényi divergence, the use of double-precision arithmetic ensures security only up to approximately 2^{47} queries.

Of course, this represents the worst-case scenario. The median and rounded average of the exponents of the combined error is 43, indicating the combined error is typically below 2^{-43} . While this is better than the worst-case bound of 2^{-37} , it is still worse than the required 2^{-46} . Applying Equation 1 again, this average-case error implies that the scheme is secure for 2^{59} queries, which is still significantly below the required 2^{64} queries. Fortunately, there are currently no known attacks that can exploit these error values introduced by the floating-point arithmetic. Moreover, it is unclear whether the existing proof is tight. For example, early precision analysis of Klein’s sampler based on standard deviation suggested much higher precision requirements than those later established using Rényi divergence. It is possible that future work may find tighter bounds or alternative proof techniques. Nonetheless, based on current knowledge, the double-precision floating-point standard does not offer sufficient precision to ensure provable security of the Gaussian sampler at a security level of $\lambda = 256$ with an allowed query bound of $q_s = 2^{64}$, and is therefore not complying with the NIST requirements for Post-Quantum signature schemes. One way to get more precision, especially for constrained devices with only a single-precision FPU, is to use TW algorithms. How this can be implemented for Falcon will be discussed next.

4 Implementation

In this section, our novel implementation of Falcon using the Triple-Word Floating-Point (TW) technique is discussed. Section 4.1 discusses the exponent range needed to represent all floating-point values that can appear in the Falcon signature scheme and shows that TW implemented with the single-precision standard will not result in issues. Then in Section 4.2 we present the new triple-word arithmetic that was not covered in [FMP19], but is required to allow us to perform all Falcon operations in triple-word form. Lastly, we show in Section 4.3 that the resulting implementation can be made constant time. Multiple TW algorithms from [FMP19] have a “fast” version with higher error. Thanks to the added precision of TW, these faster algorithms can be used for all Falcon operations, as verified in Section 5.2.

Our implementation of Falcon using TW arithmetic is available at [TWF]. This repository includes both C and ARM assembly code for all TW operations, which can be selected via a macro in the Falcon code. The code is a modification of the latest Falcon implementation [Por25b]. Falcon with TW passes all tests in the code, except for the floating-point tests due to precision discrepancies when rounding from a TW to a double.

4.1 Exponent Range in Falcon

Recall, the exponent value is the e in the floating-point formula $(-1)^s \times 1.d_0d_1\dots d_{p-2} \times 2^e$.

Falcon Exponent Range in Practice To determine the exponent range used in Falcon in practice, we collected its frequency using 100 million signature generation runs for both Falcon-512 and Falcon-1024, each using a new key and a new random message. The result for Falcon-1024 is shown in Figure 4.

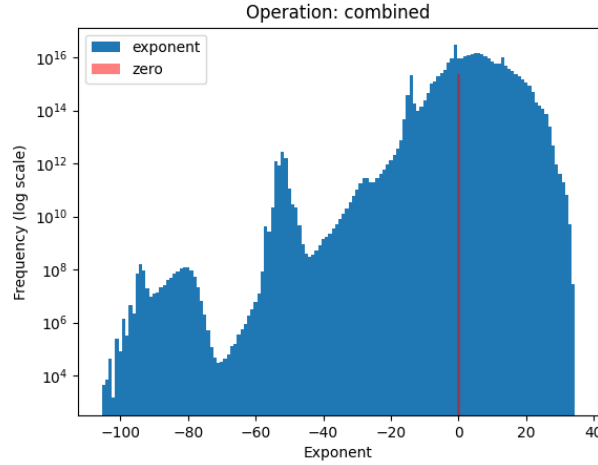


Figure 4: Frequency of each floating-point exponent in Falcon-1024 signatures (log scale), based on 100 million runs. The red line indicates the zero, which does not have a meaningful exponent in this context.

The minimum and maximum exponent values of the floating-point inputs for each operation, as well as the occurrence of zero values, are shown in Table 4 for both Falcon security levels.

Table 4: Observed exponent range and zero occurrence in floating-point inputs for each operation in 100 million Falcon-512 and Falcon-1024 runs.

Operation	Falcon-512			Falcon-1024		
	Min	Max	Zero	Min	Max	Zero
addition	-103	33	Y	-105	34	Y
multiplication	-103	33	Y	-105	34	Y
div numerator	0	0	N	0	0	N
div denominator	77	18	N	6	19	N
square-root	13	14	N	13	14	N

From these figures, it is evident that the exponent range in Falcon is quite large. For Falcon-512, the smallest observed exponent is -103, while the largest is 33. In the case of Falcon-1024, the range is slightly wider, spanning from -105 to 34. Notably, this implies

that the range of floating-point values used in both Falcon parameter sets exceeds that of 64-bit integers.

As shown in Table 4, the wide range of exponent values primarily arises in addition and multiplication operations. In contrast, the exponent ranges for division and square root are significantly narrower and entirely positive.

Formal Exponent Range Analysis A formal analysis of the exponent range was conducted by [Hwa23], using the CryptoLine modeling tool [cry, LLS⁺24]. [Hwa23] analyzed the floating-point computations in the FFT for polynomials appearing in Falcon. Relying on the assumption that all input coefficients of polynomials lie within the range $[-2^{15}, 2^{15}]$, they show that all non-zero intermediate floating-point values have absolute values within the interval $[2^{-476}, 2^{27}(2^{52} + 605, 182, 448, 294, 568)]$. This corresponds to an approximate exponent range of $[-476, 79]$.

Single-Precision Exponent Range Analysis The exponent range of the single-precision floating-point format used by TW is $[-126, 127]$. While the positive exponent range fits within the bounds proven by [Hwa23], the negative range is much smaller than the required -476 . In contrast, the double-precision format, with a range of $[-1026, 1027]$, fully accommodates the formal exponent range.

However, we’ve shown the lowest exponent observed in practice is far from the theoretical lower bound. The theoretical range assumes the full input domain of $[-2^{15}, 2^{15})$ for polynomial coefficients entering the FFT. However, because of the distributions of the private key coefficients, the practical range is smaller. After running 100 million Falcon-1024 instances, the lowest exponent encountered was only -105 . This is well above the theoretical minimum. The polynomial range of Falcon is discussed in more detail in subsection 4.2.1.

While -105 is still within the single-precision representable range, TW represents values using three components, where the second component has an exponent at least 24 less than that of the first. As a result, the second and third components may fall outside the representable range of single-precision floats. Additionally, there can be rarely occurring outliers that have even lower exponents.

To understand how to overcome this challenge, it is important to understand what happens when a floating-point exponent becomes too small. At an exponent of -126 , the implicit leading 1 in the mantissa disappears, and the number is represented in a subnormal format. This allows for smaller values to be represented by shifting the leading 1 further into the mantissa, at the cost of reduced precision. This continues until the smallest representable subnormal value, 2^{-149} , which has only a single bit of precision. If the value drops even further, the value underflows and is rounded down to zero.

In the case of division, it could pose a problem if the underflowed value ends up as the denominator. However, based on the observed exponent values for division, this scenario does not occur in practice. It is also important to note that the formal exponent range evaluation was limited to the FFT computation, which involves only addition and multiplication operations. Division and square root operations were not included in that analysis. As such, the formal bounds do not fully capture the behavior of the entire Falcon computation pipeline, and just focuses on the parts where the range is largest.

For all other operations, underflow introduces a very small absolute error, on the order of 2^{-149} . Such errors are negligible and do not significantly affect the results of multiplication or addition, where the absolute error is typically dominated by the largest values involved in the computation. This phenomenon can also occur with normal floating-point addition within the single-precision exponent range. For instance, a single-precision floating-point addition of 2^{-100} and 2^{-76} results in 2^{-76} , with an absolute error of 2^{-100} due to limited precision (24-bit). When floating-point values with significantly different exponents are

added together, the term with the smaller exponent has progressively less influence on the result as the difference in exponents increases, eventually having no influence at all. As demonstrated in Section 3, such small floating-point errors do not pose security issues.

4.2 Additional Algorithms

In addition to the arithmetic operations described in [FMP19], several supplementary algorithms are needed to fully replace floating-point arithmetic in Falcon with TW. These include conversions between TW and integer values in both directions, supporting multiple rounding modes: “round” (to nearest, ties to even), “floor” (toward minus infinity), and “truncate” (toward zero), as detailed in Sections 4.2.3–4.2.5. We will refer to these functions as *conversion functions*.

When operations are computed over values that fall within $[-2^{24}, 2^{24}]$, the range exactly representable by a single-precision floating-point value, a simpler, faster conversion algorithm can be used. Determining whether this is possible depends on multiple inputs, including the parameter μ . Parameter μ will be discussed in Section 4.2.1, and the other parameters will be discussed in detail for each new TW algorithm.

Falcon also requires efficient algorithms for additional operations, or combinations of operations, that can be implemented more efficiently through custom algorithms than using the basic TW algorithms. These are discussed in Section 4.2.6.

4.2.1 Precision in Gaussian Sampling

To determine whether the input range of the conversion functions is within $[-2^{23}, 2^{23}]$, we analyze the range of parameter μ . Recall μ represents the mean of the Gaussian sampler used at each leaf in the ffSampling algorithm (see Section 2.1). It is also denoted as c when discussing the error values in Section 3.

While other input sizes are discussed in their respective sections, μ appears in multiple conversion functions and warrants separate consideration. The values of μ originate from the preimage vector:

$$\mathbf{t} = \left(-\frac{1}{q} \cdot \text{FFT}(c) \odot \text{FFT}(F), \frac{1}{q} \cdot \text{FFT}(c) \odot \text{FFT}(f) \right),$$

which is recursively split down the Falcon tree until only single-coefficient polynomials remain at the leaves. For the leftmost leaf, μ is equal to the leaf coefficient. For subsequent leaves, μ is computed by combining the sampled value from the previous leaf with the ℓ value from the parent node, derived from the **LDL*** decomposition. As the algorithm proceeds rightward in the tree, ℓ values from higher levels are combined with accumulated sampled and target values, making an exact analytical range for μ difficult to obtain.

The range of the coefficients of $\mathbf{t}$ is known: coefficients of c (from the message hash) lie in $\mathbb{Z}_q$, and after scaling by $\frac{1}{q}$, they lie in $(-1, 0]$ or $[0, 1)$. These are then multiplied by F and f , respectively, resulting in

$$\sum_{i=1}^n \sum_{j=1}^n c_i \cdot F_j \quad \text{and} \quad \sum_{i=1}^n \sum_{j=1}^n c_i \cdot f_j.$$

However, since F_j and f_j are symmetrically distributed around zero, positive and negative terms largely cancel and the overall magnitude remains similar to a single coefficient multiplication.

The coefficients of f and g are sampled from a discrete Gaussian $D_{\mathbb{Z}, \sigma_{f,g}, 0}$ with $\sigma_{f,g} = 1.17 \cdot \sqrt{q/2n}$, and F and G are computed from the NTRU equation, making them approximately $\mathcal{O}(\sqrt{n})$ larger than f and g [PP19]. For Falcon-1024, $\sigma_{f,g} \approx 2.866$ (mean zero), and large outliers are extremely rare due to the Gaussian tail.

The ℓ values, derived from the private key, correspond to L_{10} in the $\mathbf{LDL}^*$ decomposition:

$$\ell = L_{10} = \frac{g^*G + f^*F}{|g|^2 + |f|^2}.$$

While their exact range is complex to analyze, they are ultimately tied to the distributions of f and g and do not grow excessively.

Due to the recursive, left-to-right tree structure, sampled values on the right are involved more arithmetic operations, resulting in a wider range for μ . To still give a confident range of μ values, 10,000 Falcon signature operations were run. The resulting distribution is shown in Figure 5.

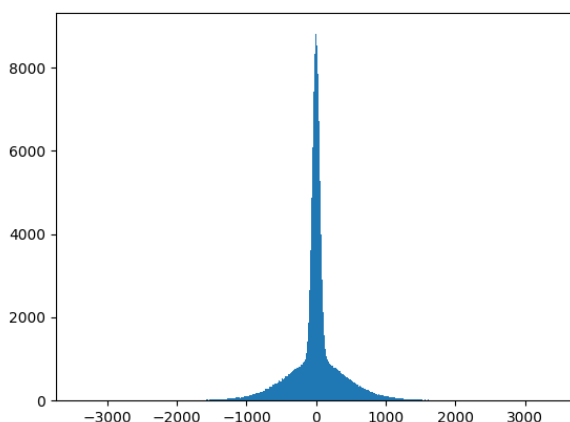


Figure 5: Range of μ from 10,000 Falcon-1024 runs.

The figure shows that μ is tightly concentrated around zero, with rare, moderate-size outliers. Across 10,000 Falcon-1024 signature runs, comprising 10.24 million leaves, and thus μ values, the smallest observed μ was -3396.7 and the largest 3405.6 . Consistent with how μ is formed, as discussed in this section, most values are small: 56.08% lie in $[-100, 100]$, and 97.76% in $[-1000, 1000]$. These ranges are orders of magnitude below $[-2^{23}, 2^{23}] = [-8\,388\,608, 8\,388\,608]$. The observed maximum is over 2400 times smaller than 2^{23} . From this test and the previous analysis, we can conclude that the probability of $|\mu|$ exceeding 2^{23} is negligible, justifying the use of more efficient rounding algorithms.

4.2.2 Input Conversion

[Por25a, Por25b] used the function `fpr_of(int i)` to convert an integer i into a floating-point value. Internally, this function computes $i \cdot 2^x$, but always for $x = 0$. This can be done more efficiently by ignoring x . To determine whether a simple and efficient implementation of `fpr_of` is sufficient, the input range should stay within $[2^{-24}, 2^{24}]$, the range of integer values that can be exactly represented in single-precision floating-point format.

Inputs of this function are either proportional to the value of z_0 from BaseSampler [PFH⁺22, Algorithm 12], which is always in the range $[0, 18]$, or to the value of μ , which is sufficiently small. Other inputs are 8-bit and 16-bit integers, and the value s from BerExp [PFH⁺22, Algorithm 14]. In the latest optimized implementation, the variable is treated as always being less than 64, since the probability of it exceeding this value is negligible [Por25a].

Therefore, we can conclude that all calls to `fpr_of` fall within $[2^{-24}, 2^{24}]$. Therefore, input values can be exactly represented using a single-precision floating-point format, and a new TW variable can be created where the first component is the input value converted to a single-precision floating-point number and the others are set to 0. This conversion can be done on the Cortex-M4 through the assembly instruction `VCVT.u32.f32 <Sd>, <Sm>` [Arm14].

4.2.3 Round

Rounding operations on TW can be implemented most efficiently if only the first component of the TW representation is relevant for determining the integer part of the rounding and the halfway point. The second component then only influences whether rounding up or down should occur when the value of the first component lies exactly at the halfway point. This means that the third component does not influence the result at all, and can be ignored. This is guaranteed when the accurate output value lies within the range $[-2^{23}, 2^{23}]$, since the largest single-precision floating-point number that can represent a halfway point exactly is $2^{23} - 0.5$. Depending on the value of the second TW component, this rounds either to 2^{23} or $2^{23} - 1$. Fortunately, the output of the rounding function is always treated as a 16-bit integer, which comfortably fits within this range.

The rounding algorithm is slightly more complex than the conversion from an integer to a TW value. First, it must be determined whether the first floating-point component, x_0 , represents a halfway point. This is done by checking whether $x_0 - \lfloor x_0 \rfloor = 0.5$. If this condition is not met, the value x_0 can simply be rounded to the nearest integer. However, if the condition is true, a more involved procedure is required. In this case, x_0 is first truncated (i.e., its fractional part is discarded). Then, depending on whether the signs of x_0 and x_1 are equal, the truncated value is either incremented or decremented. This ensures that the rounding decision is correctly influenced by the second component of the TW representation, resulting in an accurately rounded value. This results in Algorithm 3, which can be made constant-time in software easily.

Algorithm 3 RoundTW(x_0, x_1)

```

1:  $f \leftarrow x_0 - \lfloor x_0 \rfloor$ 
2: if  $f \neq 0.5$  then
3:   return round( $x_0$ )
4: end if
5:  $t \leftarrow \text{truncate}(x_0)$ 
6: if sign( $x_0$ ) = sign( $x_1$ ) then
7:   return  $t + 1$ 
8: else
9:   return  $t - 1$ 
10: end if

```

On the FPU of the Cortex-M4, there is no hardware support for rounding or flooring a floating-point value to an integer. The only available operation is truncation, which can be performed using the instruction `VCVT.f32.s32 <Sd>, <Sm>` [Arm14]. To achieve an efficient, and later, constant-time, implementation of the `floor` and `round` operations on a single-precision floating-point value, the constant-time integer-emulated double-precision versions from [Por25a] were adapted for single-precision values, resulting in an efficient and constant-time implementation.

4.2.4 Floor

This operation is only invoked on the input value μ . Therefore, from the analysis in Section 4.2.1 it can be concluded that the input range remains within $[-2^{23}, 2^{23}]$ with sufficient probability. This allows the use of the simpler and more efficient form of the algorithm.

First, the floating-point computation $x_0 + x_1$ must be computed, but with the rounding mode of the FPU set to “round toward minus infinity”. After this computation, the rounding mode is restored to the default (“round towards nearest, ties to even”) and the floor of the resulting sum can be computed. As described in Section 4.2.3, this can be done using a single-precision adaptation of the double-precision integer-emulated code.

4.2.5 Truncate

The `truncate` algorithm is the only case where, in two out of three function calls, the input value can be outside the range $[-2^{23}, 2^{23}]$, however they are always non-negative. The one instance where the input does fall within the valid range comes from the variable s in the BerExp algorithm [PFH⁺22, Algorithm 14], which, as previously explained in Section 4.2.2, remains within bounds. In the other two cases, the input value is the result of a multiplication by 2^{63} . These computations occur in the algorithm ApproxExp [PFH⁺22, Algorithm 13]. One involves the variable $x \in [0, \ln(2)]$, and the other involves $css \in [0, 1]$. When either of these values is multiplied by 2^{63} , the resulting input clearly can exceed the $[-2^{23}, 2^{23}]$ range.

To achieve optimal performance, for both cases a separate algorithm is used. The version accurate on the reduced range operates similarly to the `floor` implementation described in Section 4.2.4. First, the sum $x_0 + x_1$ is computed with the rounding mode set to “round toward zero”. Then, the result is truncated, which can be done on the Cortex-M4 with the instruction `VCVT.f32.s32 <Sd>, <Sd>`, as in Section 4.2.3.

The version accurate on the full range of the `truncate` algorithm is significantly more complex and less efficient, especially when implemented in a constant-time manner. To truncate a TW value with full precision, all three components of the TW representation must be combined into a high-precision bit string. Based on the exponent of the leading component, a cut-off point is determined in this bit string to extract the truncated integer value of the full TW value (x_0, x_1, x_2) . The most challenging part of this process is constructing the bit string with sufficient precision. A key complication arises from the implicit leading 1 in the IEEE 754 floating-point representation. Due to this, the exponent of x_1 is at least 24 higher than that of x_0 , depending on where the next 1-bit appears in the precision bits. This introduces intermediate zero bits between x_0 and x_1 , and similarly between x_1 and x_2 . Subnormals in the floating-point representation would occur if x_0 were -127 . Because the input values are multiplied by 2^{63} , subnormal representations do not occur in x_0 . Subnormals may occur in x_1 or x_2 , but if these components are that small, they do not affect the final integer result, and any inaccuracies in handling them therefore are irrelevant. Therefore, the truncate algorithm does not have to take subnormals into account.

Another complexity arises from the possibility that the signs of x_0 , x_1 , and x_2 may differ. As a result, the bits cannot simply be concatenated with intermediate zeros. Instead, they must be added or subtracted at the correct positions based on their signs. To construct the high-precision bit string, three 32-bit integers are used, with only the lower 31 bits of each being utilized. The 32nd bit is reserved for carry handling. This setup allows for a total of 93 precision bits to be represented. Despite the TW format potentially offering more than 72 bits of precision (plus intermediate zeros), the final result of the algorithm always remains below 2^{63} due to the input range. Therefore, any additional precision beyond this point does not influence the truncated result.

Due to the varying number of intermediate zero bits between the components of the TW representation, it must be determined in which of the three 32-bit integers the precision bits of each component should be added or subtracted. In some cases, the bits may span across two integers. This results in many possible cases, which should be handled in constant-time to ensure timing side-channel resistance. Once all cases are correctly handled and the high-precision bit string is constructed, a simple cut can be made at a position determined by the exponent of x_0 . Since the input to this algorithm is always positive, this cut directly yields the truncated integer value of the input.

4.2.6 Additional Algorithms

The last missing algorithms needed to implement Falcon with TW are division and multiplication by 2^e , combined addition/subtraction, and addition/multiplication between a single-precision floating-point value and a TW value.

Multiplication and division by 2^e increase and respectively decrease each exponent by e , by multiplying each component with the exactly representable floating-point value 2^e . While direct bitwise exponent manipulation is possible, on platforms such as the Cortex-M4 floating-point multiplication is a single-cycle operation, making this approach both simple and efficient.

Implementing the `add_sub` algorithm requires special care. In the integer-emulated Falcon implementation, addition and subtraction on the same input are merged into a single efficient algorithm. This is mostly not possible when using TW representation, since the merging step obscures which inputs need negation. The only viable optimization is to merge inputs once in a way that serves both operations. Furthermore, to achieve optimal performance on the Cortex-M4 custom inline assembly is required to efficiently invoke the algorithm, similar to the approach used in the integer-emulated implementation [Por25a]. This way the stack does not have to be used as all inputs and outputs fit in registers, which is not possible under the standard C calling conventions.

Lastly, we designed additional TW algorithms aimed at improving performance. Where possible, our implementation calls the addition and multiplication algorithms with one input being a full TW value and the other a result of the `fpr_of` function. As described in Section 4.2.2, this function produces a TW value with one non-zero and two zero components. In such cases, $x = (x_0, x_1, x_2)$ and $y = (y_0, 0, 0)$, allowing optimizations by skipping operations on zero components. For addition, only four values must be merged instead of six, and both `vec_sum` and `vseb` reduce to four inputs while still producing the required outputs. For the multiplication operation, [FMP19] already presented an algorithm using a double-word and TW input; we extended the same technique to a single-precision floating-point input and a TW.

4.3 Constant-Time Implementation

TW algorithms from [FMP19] are not constant time and need to be revised. The notion of constant-time used in this work follows the definition provided by [Por25a]. We make the assumption that all computational instructions executed by the FPU, including division (`vdiv`) and square root (`vsqrt`), are assumed to run in constant-time. These two instructions typically execute in 14 cycles, except when no further floating-point instructions follow, in which case they complete in a single cycle. However, since the instruction sequence is fixed and does not vary with input, the timing remains constant.

All custom assembly code written for the Cortex-M4F adheres strictly to these rules. While the C code is written to follow constant-time principles at the source level, the compiler may still introduce branches or instructions whose execution time depends on input values, potentially leading to timing side channels. Therefore, only the implementation using assembly functions on the Cortex-M4F can be confidently considered constant-time.

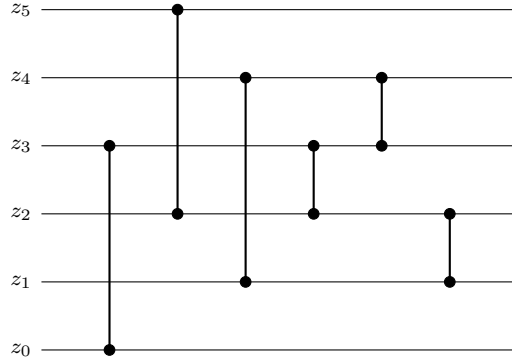


Figure 6: Sorting network of size six, optimal for the Merge algorithm.

We achieve conditional execution in constant-time by evaluating both branches and selecting the correct result using bitmasks. A mask of all 1s or all 0s is computed from the condition, and results are combined with bitwise operations. This avoids data-dependent branching.

For floating-point values, constant-time checks are obtained by reinterpreting them as integers. Since zero has the same bit pattern in both domains, masking logic can be applied after a reinterpretation of the floating-point value as an integer. This allows operations such as “ $x \neq 0$ ” to be performed without branching.

The *Merge* step requires special care, since its naive implementation relies on multiple conditional checks. Instead of replacing each comparison with masking logic, a sorting network is used. Sorting networks apply a fixed sequence of comparisons and swaps independent of the input, making them inherently constant-time [Bat68]. For the case of merging two sorted triples, a brute-force search identified an optimal network with six comparators shown in Figure 6.

Each comparator requires a constant-time swap. Since the Cortex-M4 lacks a conditional swap instruction, we design a masking-based swap: absolute values are compared using integer arithmetic, producing a mask from the sign of their difference. The swap is then carried out with XOR operations masked by this bit string. This ensures the correct ordering while avoiding data-dependent control flow. On the Cortex-M4, this can be implemented efficiently with as few as seven instructions, or further optimized using the DSP SEL instruction.

5 Results

This section presents performance results, measured in CPU cycles, for Falcon implemented using integer-emulated double-precision floating-point arithmetic from [Por25a] and Falcon implemented with TW. Additionally, we also provide the resulting error values of the Gaussian sampler when TW is used, and investigate any security implications of our approach.

5.1 Performance

All benchmarks are conducted on a Nucleo-L4R5ZI board, which also serves as the default benchmarking platform for the PQM4 library [STM, KPR⁺]. This board features a Cortex-M4F processor, where the ‘F’ denotes the inclusion of a 32-bit FPU. The Cortex-M4 is based on the ARMv7-M architecture [Arm14], and its technical specifications are detailed in the corresponding reference manual [Arm10]. Like in the integer-emulated implementation,

Table 5: Cycle count of arithmetic and conversion operations, for integer emulation and TW.

Arithmetic	add	add_f	add/sub	mul	mul_f	reci	div	sqrt
Integer Em.	80	80	100	45	45	460	460	612
TW	171	99	297	76	59	144	223	244
Conversion	fpr_of	rint	floor	trunc fast	trunc acc			
Integer Em.	34	59	33	32	32			
TW	7	55	20	12	155			

all TW functions are optimized in assembly for the ARMv7-M architecture, as used by the Cortex-M4F, and fully constant-time based on the rules discussed in Section 4.3.

The benchmarks are performed by directly calling the assembly functions and measuring the number of clock cycles required for execution. This is achieved by reusing the benchmarking done by [Por25a] in combination with the benchmark framework in the PQM4 library. In summary, the cycle counter is enabled, the CPU frequency is set to 20 MHz, and the CPU cycle counter is read before and after each operation to compute the cycle count.

Results in Table 5 show that TW arithmetic is generally slower than integer-emulated floating-point arithmetic on the Cortex-M4. The core reason is that TW implementations rely, besides for some constant-time modifications, exclusively on the FPU, which provides only a small set of basic operations: addition, multiplication, division, square root, and FMA. Each of these has non-negligible latency, for example, FMA requires three cycles, and if its output is used immediately by the next instruction, an additional cycle is required. This pipeline sensitivity is common in TW code, where sequences of dependent multiplications and additions are unavoidable. Moreover, enforcing constant-time execution amplifies these costs: conditional branches must be replaced with masking operations, which require moving values between integer and floating-point registers at a cost of one cycle per move, and in many cases intermediate results must be saved to and reloaded from the stack. These factors accumulate in simple yet frequent operations such as addition and add/sub, where TW ends up several times slower.

By contrast, integer-emulated arithmetic can take advantage of the ARM ALU and DSP instruction set, which is far richer and better optimized for multi-word integer arithmetic. Instructions such as `UMLAL` perform $32 \times 32 \rightarrow 64$ multiplication with accumulation in a single cycle. In addition, shifts can be fused into arithmetic instructions at no extra cost, and conditional moves can be realized using the `SEL` instruction, though this can also be used in the constant-time Merge algorithm for TW. These capabilities allow integer-emulated implementations to align mantissas, perform multi-word additions/multiplications, and implement conditional swaps at very low cost, while there is also no cycle penalty when the output of an instruction is used by the next one, like for the FPU. As a result, the frequent basic operations needed in Falcon, such as addition, subtraction, and multiplication, are consistently faster in the integer-emulated implementation, despite the apparent simplicity of the TW algorithms when expressed at the mathematical level.

While TW arithmetic is disadvantaged for frequent basic operations, it gains an advantage in more complex functions such as reciprocal, division, and square root. In the integer-emulated implementation, these operations require iterative bit-by-bit methods over multi-word integers, leading to hundreds of cycles of work. In contrast, the TW versions use a Newton–Raphson iteration approach, which results in a simpler algorithm that can use the single-precision square root and division operation, and use TW multiplication and FMA for other parts of the algorithm. As a result, TW implementations of division and square root are several times faster than the corresponding integer-emulated algorithms, even though they are slower for addition and multiplication.

What does this mean for a full Falcon signature generation? The results shown in Table 6 show that the TW implementation is still slower by a factor of approximately 1.8. This slowdown is primarily due to the increased usage of addition and multiplication operations, which are more frequent and faster with integer emulation.

Multiplication is the most frequently used operation, followed by addition and combined addition/subtraction. Table 1 from Section 2.1 shows, for Falcon-512, how often each operation is invoked. Although TW excels in operations like division and square root, their infrequent use does not offset the slowdown from addition and multiplication. Additionally, the TW implementation uses more registers, as each floating-point value uses three values, instead of 2 in the integer-emulation, leading to increased overhead from stack operations. Despite these factors, the TW implementation offers higher precision, and next, in Section 5.2, the effect on the security will be discussed.

Table 6: Average cycle count of Falcon signature generation with integer-emulated and TW arithmetic, assembly-optimized for Cortex-M4F, over 100 runs on the Nucleo-L4R5ZI.

Falcon-	Integer emulated	TW
512	22,493,190	40,684,889
1024	48,477,027	89,534,446

5.2 Gaussian Sampler Floating-Point Errors

In this section, the error values δ_μ and δ_σ resulting from running Falcon with TW arithmetic are discussed. Further details on the importance of these error values for precision are provided in Section 3. The code used for the error evaluation in Section 3.2 and in this section is available at [TWF].

To compute the error values, the methodology described in Section 3.2 is followed. The MPFR library is used for high-precision arithmetic and error computation [FHL⁺07]. For the TW arithmetic, the fast, constant-time C implementations of all relevant functions are used. The exponent values of the combined error ($\delta_\mu + \delta_\sigma$) are shown in Figure 7. These results demonstrate that the higher precision of TW, even when using faster and less precise algorithms, yields significantly lower error values compared to the standard double-precision implementation, as illustrated in Figure 3. As discussed in Section 3.1, the combined error should remain below 2^{-46} . This threshold is not met by the double-precision implementation, which produces errors as low as 2^{-37} with a median of 2^{-43} . In contrast, the TW implementation performs substantially better, with the smallest observed combined error being 2^{-57} and a median of 2^{-62} . The use of double-word arithmetic is not investigated, because it provides only 48 bits of precision. This is less precise than the double-precision floating-point standard, which itself does not meet the required error bounds. Like we discussed in Section 3.2, empirical testing cannot prove that a bound will never be exceeded. Though, up to 11 lower exponents are allowed before the proven bound is exceeded, which if we look at the exponential reduction of frequency for the lower exponents, gives us high assurance that this will never happen. These results confirm that Falcon implemented with TW provides more than sufficient precision for a provably secure Gaussian sampler.

6 Conclusion

In this work we demonstrated that employing the double-precision standard in Falcon yields observed error values of at most 2^{-37} . For a security level of $\lambda = 256$, this restricts the number of allowed signing queries to 2^{47} in order to maintain a Gaussian sampler implementation that is indistinguishable from an exact Gaussian distribution. This falls

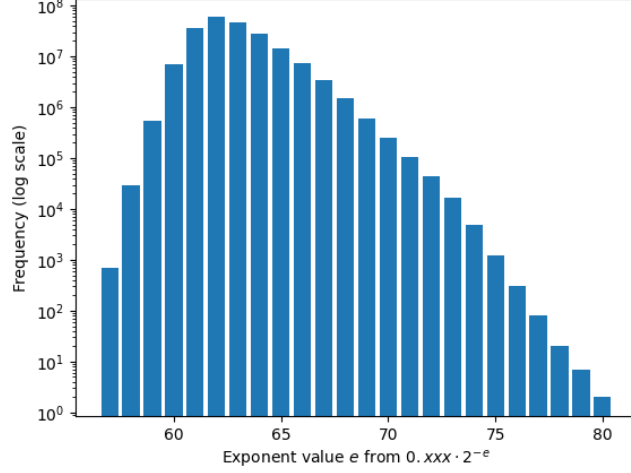


Figure 7: The frequency of each exponent of the combined error value ($\delta_c + \delta_\sigma$) for Falcon-1024 implemented with TW from 100,000 signature generation runs.

short of the 2^{64} signing queries required by NIST. In contrast, Section 5.2 shows that implementing Falcon with triple-word arithmetic achieves error values of at most 2^{-57} , which is well within the proof requirement of 2^{-46} . This ensures that the Gaussian sampler remains secure and meets the necessary cryptographic standards. We show that the exponent range of single-precision floating-point values is sufficient for all values encountered in Falcon, enabling its use with TW for its implementation.

We note that our novel method to increase the precision is not the only possible approach. The higher precision could also be reached with integer emulation, by creating a custom standard that, like TW, uses three words to contain sufficient precision. However, for such a custom standard, the required arithmetic operations are not straightforward, and it is not known if this would result in a faster implementation. Additionally, for devices with a double-precision FPU, the use of double-word precision could be investigated, to yield the combined precision of two double-precision values. We leave both open as future work.

In this paper, we also present novel TW algorithms, and as a result, enable a constant-time implementation of Falcon using TW that can efficiently leverage the single-precision FPU available on processors such as the Cortex-M4F. We show that performance comparison of Falcon on a Nucleo-L4R5ZI board with a Cortex-M4F, comparing an integer-emulation and TW implementation, incurs a slowdown of less than 85%. For use cases where such a slowdown is acceptable, our work enables a highly secure Falcon implementation, which, verified through testing, contains a provably secure Gaussian sampler implementation available for a large class of constrained devices.

Acknowledgements We would like to thank Thomas Pornin and Thomas Prest from the Falcon team for helpful discussions and clarifications regarding Falcon and preliminary results. We further thank Andreas Hülsing for valuable discussions and insights during the project. Lastly, we would like to thank William R. Duro and Markus Schönauer for preliminary work and discussions.

Part of this work was supported by the SINFONIA project, which has received funding from the Recovery and Resilience Facility (RRF) as the centrepiece of NextGenerationEU via the Austrian Research Promotion Agency (FFG) and Austria Wirtschaftsservice

Gesellschaft mbH (aws) in the frame of the IPCEI ME/CT – Important Project of Common European Interest on Microelectronic and Communication Technologies, under FFG project No. 917423 and AWS project No. P2431566. The work described in this paper has also been supported by the German Federal Ministry of Education and Research (BMBF) through the project EASEPROFIT (16KIS2126).

References

- [Arm10] Arm Ltd. Cortex-M4 technical reference manual. <https://developer.arm.com/documentation/ddi0439/be>, 2010.
- [Arm14] Arm Ltd. ARMv7-M architecture reference manual. <https://developer.arm.com/documentation/ddi0403/latest>, 2014.
- [Arm25] Arm Ltd. Arm Cortex-M processor comparison table. <https://developer.arm.com/documentation/102787/latest>, 2025. Accessed: 2025-09-23.
- [Bab86] László Babai. On lovász’ lattice reduction and the nearest lattice point problem. *Combinatorica*, 6:1–13, 1986.
- [Bat68] K. E. Batchier. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, AFIPS ’68 (Spring), page 307–314, New York, NY, USA, 1968. Association for Computing Machinery.
- [BDF⁺11] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 41–69. Springer, Berlin, Heidelberg, December 2011.
- [cry] CryptoLine: A tool and language for verification of low-level implementations of mathematical constructs. <https://github.com/fmlab-iis/cryptoline>.
- [DP16] Léo Ducas and Thomas Prest. Fast fourier orthogonalization. In Sergei A. Abramov, Eugene V. Zima, and Xiao-Shan Gao, editors, *Proceedings of the ACM on International Symposium on Symbolic and Algebraic Computation, ISSAC 2016, Waterloo, ON, Canada, July 19–22, 2016*, pages 191–198. ACM, 2016.
- [FHL⁺07] Laurent Fousse, Guillaume Hanrot, Vincent Lefèvre, Patrick Pélicier, and Paul Zimmermann. Mpfir: A multiple-precision binary floating-point library with correct rounding. *ACM Trans. Math. Softw.*, 33(2):13–es, June 2007.
- [FMP19] Nicolas Fabiano, Jean-Michel Muller, and Joris Picot. Algorithms for triple-word arithmetic. *IEEE Transactions on Computers*, 68(11):1573–1583, 2019.
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 197–206. ACM Press, May 2008.
- [GVL13] Gene H. Golub and Charles F. Van Loan. *Matrix Computations - 4th Edition*. Johns Hopkins University Press, Philadelphia, PA, 2013.
- [HBD⁺22] Andreas Hülsing, Daniel J. Bernstein, Christoph Dobraunig, Maria Eichlseder, Scott Fluhrer, Stefan-Lukas Gazdag, Panos Kampanakis, Stefan Kölbl, Tanja Lange, Martin M. Lauridsen, Florian Mendel, Ruben Niederhagen, Christian Rechberger, Joost Rijneveld, Peter Schwabe, Jean-Philippe Aumasson, Bas Westerbaan, and Ward Beullens. SPHINCS⁺. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.

- [Hwa23] Vincent Hwang. Formal verification of emulated floating-point arithmetic in falcon. In Junji Shikata and Hiroki Kuzuno, editors, *IWSEC 23*, volume 14128 of *LNCS*, pages 125–141. Springer, Cham, August 2023.
- [IEE19] IEEE standard for floating-point arithmetic. *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pages 1–84, 2019.
- [Kle00] Philip Klein. Finding the closest lattice vector when it’s unusually close. In *Proceedings of the Eleventh Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA ’00, page 937–941, USA, 2000. Society for Industrial and Applied Mathematics.
- [KPR⁺] Matthias J. Kannwischer, Richard Petri, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Post-quantum crypto library for the ARM Cortex-M4. <https://github.com/mupq/pqm4>.
- [LDK⁺22] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [LLS⁺24] Li-Chang Lai, Jiaxiang Liu, Xiaomu Shi, Ming-Hsien Tsai, Bow-Yaw Wang, and Bo-Yin Yang. automatic verification of cryptographic block function implementations with logical equivalence checking. In *Computer Security - ESORICS 2024: 29th European Symposium on Research in Computer Security, Bydgoszcz, Poland, September 16-20, 2024, Proceedings, Part IV*, pages 377–395, Berlin, Heidelberg, 2024. Springer-Verlag.
- [NIS] National Institute of Standards and Technology. *Post quantum cryptography standardization*. <https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization>.
- [NIS23a] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard (ML-DSA)*, 2023. Federal Information Processing Standards Publication 204 (Initial Public Draft), <https://doi.org/10.6028/NIST.FIPS.204.ipd>.
- [NIS23b] National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*, 2023. Federal Information Processing Standards Publication 203 (Initial Public Draft), <https://doi.org/10.6028/NIST.FIPS.203.ipd>.
- [NIS23c] National Institute of Standards and Technology. *Stateless Hash-Based Digital Signature Standard (SLH-DSA)*, 2023. Federal Information Processing Standards Publication 205 (Initial Public Draft), <https://doi.org/10.6028/NIST.FIPS.205.ipd>.
- [PFH⁺22] Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. FALCON. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [Por19] Thomas Pornin. New efficient, constant-time implementations of Falcon. Cryptology ePrint Archive, Report 2019/893, 2019.

- [Por25a] Thomas Pornin. Falcon on ARM cortex-M4: an update. Cryptology ePrint Archive, Report 2025/123, 2025.
- [Por25b] Pornin, Thomas. c-fn-dsa. <https://github.com/pornin/c-fn-dsa>, 2025. Commit: 96e3b92c4f1bafa23545388a207954fef1a74c90.
- [PP19] Thomas Pornin and Thomas Prest. More efficient algorithms for the ntru key generation using the field norm. In Dongdai Lin and Kazue Sako, editors, *Public-Key Cryptography – PKC 2019*, pages 504–533, Cham, 2019. Springer International Publishing.
- [Pre17] Thomas Prest. Sharper bounds in lattice-based cryptography using the Rényi divergence. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 347–374. Springer, Cham, December 2017.
- [Pri91] D.M. Priest. Algorithms for arbitrary precision floating point arithmetic. In [1991] *Proceedings 10th IEEE Symposium on Computer Arithmetic*, pages 132–143, 1991.
- [SAB⁺22] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [STM] STMicroelectronics. NUCLEO-L4R5ZI-P – STM32 Nucleo-144 development board with STM32L4R5ZI MCU. <https://www.st.com/en/evaluation-tools/nucleo-l4r5zi-p.html>.
- [TWF] Twfalcon code, written as part of this paper. <https://github.com/NXP-Research/TWFalcon>.
- [Yiu16] Joseph Yiu. Cortex-M for beginners: An overview of the Arm Cortex-M processor family and comparison. Technical report, Arm Ltd., 2016. White Paper (final v3).